Implementación de Presto — guía técnica del código

1. Vista general
2. Organización del código y capas
3. Backend, app por app
4. Base de datos e integridad
5. API
6. Configuración y seguridad
7. Frontend
8. Infraestructura, deploy y CI
9. Testing y calidad
10. Decisiones de diseño (ADRs)
11. Qué está implementado y qué falta
12. Mapa rápido y comandos

Implementación de Presto — guía técnica del código

Este documento describe **qué está implementado y dónde**, a nivel de código: modelos, servicios, selectores, endpoints, invariantes, configuración, frontend, infraestructura y pruebas. No reemplaza a `architecture.md` (visión de arquitectura) ni a `AGENTS.md` (reglas del proyecto): los complementa con el detalle de la implementación real.

> **Estado:** demo pública con dinero simulado. **Fecha:** 2026-10-23 · rama `main` . **Base de código:** backend Django en la raíz + front React en `frontend/` .
>
> **Versiones para compartir:** PDF · Word.

1. Vista general

Área	Tecnología / versión
Lenguaje backend	Python **3.14**
Framework	Django **6.1.2**
API	Django REST Framework **3.18.3**
Auth	`djangorestframework-simplejwt` **5.5.1** (+ `token_blacklist`)
OpenAPI	`drf-spectacular` **0.30.0**
Base de datos	PostgreSQL **16**
Cache / broker	Redis 7 + Celery **5.6.3**
Frontend	React **18.3** + Vite **5.4** + TypeScript **5.6** + Tailwind 4
Tests	pytest + pytest-django + pytest-cov + factory_boy
Calidad	ruff + import-linter (32 contratos)
Infra	Docker multi-stage, Render, Vercel, GHCR, GitHub Actions

Números actuales: 13 apps de dominio, 17 migraciones, **800 tests**, 5 jobs de CI y **37 tests unitarios del front** (más specs E2E de Playwright).

2. Organización del código y capas

```
apps/
  core/          utilidades transversales
  identity/      usuarios, JWT, perfil, KYC
  ledger/        libro mayor de doble entrada
  payments/      intención de pago + webhooks
  offers/        ofertas de préstamo
  matching/      búsqueda y comparación (solo lectura)
  loans/         préstamos y cuotas
  investments/   plazos fijos
  budgets/       presupuestos y gastos
  reports/       reportes de solo lectura + CSV
  notifications/ avisos por email
  assistant/     Presti (mercado, noticias, legal, chat por reglas)
  demo/          seed_demo y comandos de demo
config/          settings, urls, celery, wsgi/asgi
frontend/        SPA React
deploy/          entrypoint, Caddy, backup
scripts/         setup, servicios, seed, smoke
tests/           suite end-to-end
```

Reglas que el código respeta y que `import-linter` verifica:

- **`views.py` finas** → la lógica de escritura vive en `services.py` y la de lectura en `selectors.py`. Los serializers solo validan/formatean.
- **Entre apps solo se usan `services.py` y `selectors.py`**, nunca modelos, vistas ni serializers de otra app.
- **Sin ForeignKey entre apps de dominio**: las referencias cruzadas son `UUIDField`. Las FK internas sí existen (`Position.product` , `Budget.category`).
- **Dinero siempre `Decimal`**, con un único redondeo en `core/money.py` .
- **Idempotencia obligatoria** en las operaciones que mueven dinero.

3. Backend, app por app

3.1 `core` — utilidades transversales

Archivo	Contenido
`models.py`	`BaseModel`: UUID PK, `created_at`, `updated_at`
`money.py`	`round_two`, `money`, `monthly_rate_from_annual_tna`, `french_installment`, `simple_interest`, `french_total_interest`, `french_total_cost`, `effective_annual_rate`
`exceptions.py`	`DomainError`, `InsufficientFunds`, `InvalidAmount`, `KYCNotCompleted`
`pagination.py`	`StandardPagination`: 100 por página, `page_size` configurable, máximo 1000
`exception_handler.py`	Formato uniforme `{"error": {"code", "message", "details"}}`
`middleware.py`	`DemoModeHeaderMiddleware` (cabecera `X-Presto-Mode: demo`) y `HealthCheckMiddleware` (responde `/health/` **antes** de validar `Host` y del redirect TLS)
`openapi.py`	Hooks de drf-spectacular: errores por operación, CSV de reportes, respuestas nullables, cabecera de demo
`views.py`	`/health/` (200/503 según la base) y `/api/v1/meta/` (`mode`, `simulated_money`, `compliance`)

`HealthCheckMiddleware` es clave en plataformas: el health check llega con la IP interna o un `Host` no permitido. Sin él, Django responde 400/301 y el servicio nunca queda sano.

3.2 `identity` — usuarios, sesión, KYC

Modelos

- `User`: login por **email** (sin `username`), `email_verified_at` y la propiedad `email_verified`; `UserManager` propio.
- `Profile`: teléfono, fecha de nacimiento, país ISO-3166 alpha-2.
- `KYCVerification`: estado `pending|approved|rejected`, `documents` JSON, `requested_at`, `resolved_at`, `resolved_by_user_id`. Restricción: **una sola verificación `pending` por usuario**.

Servicios (`apps/identity/services.py`)

- `register_user`: email único, valida contraseña con los validadores de Django, crea el perfil y **envía el email de verificación**.
- `submit_kyc`, `resolve_kyc` (aprobar/rechazar, solo staff), `update_profile`.
- Cookies: `set_refresh_cookie`, `clear_refresh_cookie`, `get_refresh_token`.
- Reseteo: `request_password_reset` (siempre responde igual, no filtra emails), `confirm_password_reset` (valida token, cambia clave y **blacklistea todos los refresh tokens vigentes**).
- Verificación: `send_email_verification`, `confirm_email_verification` (idempotente).
- `tokens.py`: `encode_uid` / `decode_uid` y un generador de token de verificación que **no incluye `last_login`** (así el enlace sigue válido si el usuario inicia sesión antes de confirmar).
- `emails.py`: armado de los correos con enlaces a `FRONTEND_URL` (`/restablecer`, `/verificar`).

Endpoints

```
POST /api/v1/auth/register/
POST /api/v1/auth/login/
POST /api/v1/auth/token/refresh/
POST /api/v1/auth/logout/
POST /api/v1/auth/password-reset/
POST /api/v1/auth/password-reset/confirm/
POST /api/v1/auth/email/verify/
POST /api/v1/auth/email/resend/
GET/PATCH /api/v1/auth/me/
GET/POST /api/v1/kyc/
GET /api/v1/kyc/{id}/
```

Sesión

- El login devuelve solo el `access`; el **refresh viaja en la cookie httpOnly** `presto_refresh` (path `/api/v1/auth/`), con `SameSite` y `Secure` configurables.
- El refresh rota el token y el anterior queda en blacklist.
- El access dura 30 minutos; el refresh, 7 días.
- `IsKycApproved` / `KycApprovedForUnsafeMethods` exigen KYC aprobado en las operaciones que mueven dinero; los `GET` solo piden sesión.
- La verificación de email es **informativa**: el gate para operar es el KYC.

3.3 `ledger` — libro mayor de doble entrada

Modelos

- `Account`: `user_id` (nulo para contrapartidas), `currency` (`ARS|USD`), `kind` (`user_wallet|custody|system`), `balance` desnormalizado.
- `LedgerTransaction`: `type` (`deposit|withdrawal|transfer|interest|fee|reversal`), `idempotency_key`, `request_hash`, `created_by_user_id`, `destination_account_id`, `occurred_at`, `description`, `reverses`.
- `LedgerEntry`: `ledger_transaction`, `account`, `direction` (`debit|credit`), `amount`, `currency`.

Servicios (`apps/ledger/services.py`)

`build_request_hash`, `create_user_wallet`, `deposit`, `withdraw`, `transfer`, `transfer_between_users`, `transfer_to_counterpart`, `transfer_from_counterpart`, `reverse_transaction` (solo staff) y helpers de bloqueo, idempotencia y validación de saldo.

Invariantes (código + PostgreSQL)

1. **Doble entrada**: `sum(debits) == sum(credits)` por transacción, garantizado por un **constraint trigger diferido** de PostgreSQL.
2. **Inmutabilidad**: triggers bloquean `UPDATE` y `DELETE` sobre `LedgerTransaction` y `LedgerEntry`. Un error se corrige con un reverso.
3. **Saldo no negativo solo en** `user_wallet`; `custody` y `system` pueden ser negativas porque representan el mundo exterior.
4. **La suma de todos los saldos de una moneda es cero**.
5. **Idempotencia por** `(usuario, clave)` **+** `request_hash`: misma clave y mismo payload devuelve la operación original; mismo key con otro payload → `409`. Ante una carrera de concurrencia se reintenta y se devuelve la original.

6. `select_for_update` para bloquear cuentas antes de mover saldo.
7. Los depósitos simulados se bloquean en producción salvo `DEMO_MODE` (system check `ledger.E001`).

Migraciones

- `0001_initial`: tablas.
- `0002_ledger_triggers`: funciones/triggers de doble entrada e inmutabilidad.
- `0003_...`: `CheckConstraint` de moneda (`ARS`/`USD`) en `Account` y `LedgerEntry`.

Endpoints

```
GET/POST /api/v1/ledger/accounts/
GET /api/v1/ledger/accounts/{id}/balance/
GET/POST /api/v1/ledger/transactions/
GET /api/v1/ledger/transactions/{id}/
```

El listado de movimientos usa **paginación por cursor** (50 por página, máximo 200) e incluye el importe con signo de la pata del usuario y su moneda.

3.4 `payments` — intención de pago y webhooks

Modelos

- `Payment`: `direction` (`deposit|withdrawal|transfer`), `amount`, `currency`, `provider` (`simulated|mercadopago`), `provider_reference` único, `status` (`pending|confirmed|failed|reversed`), `idempotency_key`, `ledger_transaction_id`, `destination_user_id`, `payload`, `error`.
- `WebhookEvent`: `provider`, `external_id`, `event_type`, `status` (`received|processed|ignored|failed`), `payment_id`, `payload`, `error`.

Servicios

`create_payment`, `confirm_payment`, `fail_payment`, `handle_webhook` y helpers de proveedor, idempotencia y movimiento de dinero. El saldo lo mueve **siempre** el `ledger`; `payments` registra la intención y su trazabilidad.

Proveedores (`providers.py`)

- `SimulatedProvider`: demo y tests.
- `MercadoPagoProvider`: llamadas HTTP a Mercado Pago y validación de la firma HMAC-SHA256 del webhook, con edad máxima (`PAYMENTS_WEBHOOK_MAX_AGE_SECONDS`).

Se elige con `PAYMENTS_PROVIDER`; Mercado Pago requiere `MERCADO_PAGO_ACCESS_TOKEN`, `MERCADO_PAGO_WEBHOOK_SECRET` y `MERCADO_PAGO_WEBHOOK_URL`.

Endpoints

```
GET/POST /api/v1/payments/
GET /api/v1/payments/{id}/
POST /api/v1/payments/webhooks/{provider}/
```

El webhook es público (lo llama el proveedor), con throttle propio y sin sesión.

3.5 `offers` — ofertas de préstamo

- Modelo `Offer`: `lender_user_id`, `amount`, `currency`, `interest_rate` (TNA anual en %), `term_months`, `requirements`, `status` (`active|expired|accepted`).
- **Publicar una oferta no reserva fondos**: la reserva/desembolso ocurre al aceptarla. El prestamista necesita KYC aprobado.
- Servicios: `create_offer`, `update_offer` (también expira), `expire_offer`, `accept_offer` (bloqueo y cambio de estado).
- Selectores: `get_offer`, `list_offers_for_lender`, `get_active_offer`, `search_active_offers`.
- Endpoints: `GET/POST /api/v1/offers/`, `GET/PATCH /api/v1/offers/{id}/`.

3.6 `matching` — búsqueda y comparación (solo lectura)

- Sin modelos.
- `search_offers`: filtros por moneda, rango de montos, tasa máxima, plazo y ordenamiento.
- `compare_offers`: devuelve cuota, interés total y costo total usando `core/money.py`.
- Endpoints: `GET /api/v1/matching/offers/`, `POST /api/v1/matching/comparisons/`.

3.7 `loans` — préstamos y cuotas

Modelos

- `Loan`: prestatario, prestamista, oferta, principal, moneda, tasa, plazo, estado (`active|paid|defaulted`), `accepted_at`, `ledger_transaction_id`, idempotencia.
- `OfferTermsSnapshot`: copia **inmutable** de las condiciones aceptadas (`snapshot_data` JSON, `is_immutable`).
- `Installment`: número, vencimiento, capital, interés, importe, estado (`pending|paid|overdue`), `paid_at`, `ledger_transaction_id`.

Servicios

- `build_installment_schedule`: amortización francesa, última cuota ajusta el capital y absorbe el redondeo.
- `accept_offer`: atómico; crea el préstamo, el snapshot y las cuotas, y **desembolsa** desde la billetera del prestamista a la del prestatario.
- `pay_installment`: solo el prestatario, **en orden**, por el importe completo; mueve el dinero y actualiza la cuota y el préstamo.
- `change_loan_status`, `mark_overdue`.
- `tasks.py`: marcar vencidos y avisar cuotas por vencer/vencidas.

Endpoints

```
GET/POST /api/v1/loans/
GET /api/v1/loans/{id}/
POST /api/v1/loans/{id}/installments/{number}/pay/
```

Terceros reciben `404` (aislamiento por partes).

3.8 `investments` — plazos fijos

- `InvestmentProduct`: tipo `fixed_term`, moneda, plazo en días, TNA anual, monto mínimo, estado (`active|inactive`). **Solo staff** los crea desde el admin.
- `Position`: usuario, producto (FK interna), importe, estado (`active|matured`), apertura, vencimiento, interés calculado, idempotencia.
- `open_position`: atómico; mueve el dinero de la billetera a la cuenta `system`; calcula **interés simple** al vencimiento.
- `settle_matured_positions`: liquida vencidas de forma idempotente; tarea diaria.
- Endpoints: `GET /products/`, `GET/POST /positions/`, `GET /positions/{id}/`.
- No hay cancelación anticipada (decisión de producto).

3.9 `budgets` — presupuestos y gastos

Modelos

- `Category`: etiqueta de gasto del usuario.
- `Budget`: categoría (FK interna), período `monthly`, límite, moneda, `alert_threshold` (%).
- `Expense`: usuario, categoría, `ledger_transaction_id`. **El importe no se guarda**: se deriva del ledger al leer.

Regla de "gastado" (ADR 0002)

- El ledger es la fuente de verdad; no se duplican movimientos.
- Transferencias entre cuentas propias e inversiones **no** cuentan como gasto.
- El desembolso de un préstamo **no** es gasto; el pago de una cuota cuenta solo por el **interés** (el capital es amortización).
- Sin categoría no hay asignación automática; nunca se suman ARS y USD.
- Alerta cuando el gasto acumulado del mes supera el umbral.

Endpoints

```
GET/POST /api/v1/budgets/
GET/PATCH /api/v1/budgets/{id}/
GET /api/v1/budgets/{id}/status/
GET/POST /api/v1/budgets/categories/
GET/PATCH /api/v1/budgets/categories/{id}/
GET/POST /api/v1/budgets/expenses/
```

3.10 `reports` — reportes y CSV

- Sin modelos ni escrituras.
- Cuatro reportes: `spending`, `loans`, `investments`, `movements`.
- Todos piden `currency` (`ARS|USD`); aceptan `from` / `to`, por defecto el mes calendario actual en UTC, máximo 12 meses.
- `?export=csv` devuelve CSV con `Content-Disposition`; hay un tope de filas.
- Endpoints: `/api/v1/reports/{spending|loans|investments|movements}/`.

3.11 `notifications` — avisos por email

- `Notification`: `user_id`, `event_type`, `reference_id`, estado (`pending|sent|failed`), `sent_at`, `error`, `attempts`.
- `NotificationPreference`: usuario + evento + `enabled`.

- Eventos: `installment_due_soon`, `installment_overdue`, `investment_settled`, `budget_alert`.
- Las apps emisoras (`loans`, `investments`, `budgets`) definen señales; `receivers.py` crea el aviso y encola el envío con `transaction.on_commit`.
- Envío por Celery (`send_notification_email`), con **3 intentos y backoff**; los fallos quedan registrados.
- Endpoints actuales: `GET /api/v1/notifications/` y `GET/PUT /api/v1/notifications/preferences/`. **No hay** endpoints de "marcar como leído" en esta versión: el modelo no tiene `read_at`.
- En demo, sin SMTP, el email sale por consola (logs del worker).

3.12 `assistant` — Presti, sin LLM

- Sin modelos.
- `market.py`: dólar (Bluelytics) y cripto (CoinGecko), con timeout, caché de 60 s y degradación a listas vacías si la fuente falla.
- `news.py` dentro de `market.py`: RSS configurable (`ASSISTANT_NEWS_FEED_URL`), hasta 8 noticias, caché de 5 min.
- `regulations.py`: **6 fichas curadas** (mutuo, entidades/BCRA, CNV, tasas, PLA/FT, consumidor) con normas, enlaces oficiales y disclaimer. No se generan con un modelo.
- `services.py`: chat por intenciones; responde saldo, cuotas, inversiones, KYC, publicar ofertas, dólar, cripto, mercado y marco legal. Si no entiende, devuelve `intent: fallback` con sugerencias. Nunca inventa.
- `selectors.py`: resúmenes del usuario usando selectores de `ledger`, `loans`, `investments`, `offers` e `identity`.
- Endpoints (auth + throttle 120/h):

```
GET  /api/v1/assistant/market/
GET  /api/v1/assistant/news/
GET  /api/v1/assistant/regulations/
GET  /api/v1/assistant/regulations/{id}/
POST /api/v1/assistant/chat/
```

3.13 `demo` — datos de demostración

- Sin modelos.
- `seed_demo` (idempotente): crea 3 personas (`ana`, `beto`, `carla`) con KYC y email verificados, depósitos, una oferta aceptada, un préstamo con una cuota pagada, un plazo fijo, un presupuesto con alerta y pagos/transferencias.
- `create_demo_superuser`: crea o actualiza el superusuario del admin desde `DEMO_SUPERUSER_EMAIL` / `DEMO_SUPERUSER_PASSWORD`.
- Comandos existentes: `settle_investments`, `mark_overdue_loans`.

4. Base de datos e integridad

- **PostgreSQL 16** en todos los entornos; los tests corren contra PostgreSQL real, nunca SQLite.
- **17 migraciones** repartidas entre identity (5), ledger (3), payments (1), offers (1), loans (2), investments (1), budgets (2), notifications (2).
- Restricciones destacadas:
  - `Account.balance >= 0` solo para `user_wallet`.
  - Moneda `ARS|USD` en cuentas y asientos.
  - Montos positivos, tasas y plazos positivos en ofertas.

    - Unicidad de idempotencia por usuario + hash.
    - Una KYC `pending` por usuario.
    - Un presupuesto por `(usuario, categoría, período, moneda)`.
    - `provider_reference` único en pagos.
- Triggers de PostgreSQL del ledger: balance diferido e inmutabilidad.
- Bloqueos `select_for_update` para préstamos, cuentas y ofertas; tests de concurrencia en las apps críticas.

5. API

- Prefijo `/api/v1/`; JSON; UUID públicos; fechas ISO 8601 UTC.
- Importes como **string decimal**; nunca `float`.
- **Errores uniformes**:

```
{"error": {"code": "validation_error", "message": "...", "details": {}}}
```

- **Paginación**:
    - estándar: `{count, next, previous, results}`, 100 por página, máximo 1000;
    - movimientos del ledger: cursor, 50 por página, máximo 200.
- `Idempotency-Key` **obligatorio** en depósitos, retiros, transferencias, aceptar oferta, abrir posición y pagos. Único por usuario; misma clave + mismo payload devuelve lo original; mismo key + otro payload → `409`.
- **OpenAPI**: `/api/schema/`, Swagger en `/api/docs/`; el esquema se versiona en `frontend/contract/openapi.yaml` y se valida en CI.
- Cabecera `X-Presto-Mode: demo` en todas las respuestas cuando `DEMO_MODE=true`.
- `/health/` responde antes de `Host` y TLS; `/api/v1/meta/` informa modo, dinero simulado y pendientes de compliance.

6. Configuración y seguridad

Toda la configuración sensible viene por entorno. Grupos principales:

Grupo	Variables
Django	`DJANGO_SECRET_KEY`, `DJANGO_DEBUG`, `DJANGO_ALLOWED_HOSTS`, `DJANGO_ENV`
Base	`DATABASE_URL` (o `POSTGRES_*`)
Redis/Celery	`REDIS_URL`, `CELERY_BROKER_URL`
Email	`EMAIL_HOST`, `EMAIL_PORT`, `EMAIL_HOST_USER`, `EMAIL_HOST_PASSWORD`, `EMAIL_USE_TLS`, `DEFAULT_FROM_EMAIL`
JWT cookie	`JWT_REFRESH_COOKIE_NAME`, `_PATH`, `_SECURE`, `_SAMESITE`
Front/emails	`FRONTEND_URL`, `PASSWORD_RESET_TIMEOUT`
CORS	`CORS_ALLOWED_ORIGINS` (sin `*`, credenciales permitidas)
Demo	`DEMO_MODE`, `LEDGER_SIMULATED_DEPOSITS_ENABLED`
Pagos	`PAYMENTS_PROVIDER`, `MERCADO_PAGO_*`, `PAYMENTS_WEBHOOK_MAX_AGE_SECONDS`, `PAYMENTS_HTTP_TIMEOUT`
Asistente	`ASSISTANT_MARKET_ENABLED`, `ASSISTANT_HTTP_TIMEOUT`, `ASSISTANT_MARKET_CACHE_SECONDS`, `ASSISTANT_NEWS_CACHE_SECONDS`, `ASSISTANT_NEWS_FEED_URL`
Legal	`LEGAL_REVIEW_ACKNOWLEDGED`
Render	`RENDER_EXTERNAL_HOSTNAME` (se suma solo a `ALLOWED_HOSTS`)

Throttles

Scope	Límite
login	5/min
register	5/hour
password reset	5/hour
email verification	5/hour
payments webhook	60/min
assistant	120/hour

Seguridad

- Contraseñas con **Argon2** primero y validadores de Django activos.
- JWT con rotación y blacklist; refresh en cookie httpOnly; logout lo invalida.
- Reseteo de contraseña sin enumeración de cuentas y con cierre de sesiones.
- CORS explícito, sin `*`, con credenciales y `idempotency-key` permitido.
- Producción: `DEBUG=False`, redirect a HTTPS, HSTS, `X-Forwarded-Proto`, cookies seguras y `SameSite=None` para front/API en dominios distintos.

- Contenedores corren como usuario `presto` (uid 1000), no root.
- Secretos solo por entorno; `.env` está en `.gitignore` y no se versiona.

7. Frontend

- **React + Vite + TypeScript + Tailwind**, mobile-first.
- Rutas:

```
/                  landing pública
/legal             alcance de la demo
/login  /registro
/recuperar  /restablecer  /verificar
/dashboard  /billetera  /pagos
/presupuestos  /marketplace  /ofertas
/prestamos  /prestamos/:id
/inversiones  /asistente  /perfil
```

- **Sesión** (`features/auth/AuthContext.tsx`): el `access` vive **solo en memoria**; el refresh se pide con `credentials: include` y la cookie httpOnly. Al abrir la app se intenta recuperar la sesión; si no hay cookie, queda deslogueado.
- **Cliente HTTP** (`shared/api/client.ts`): `fetch` con `credentials: include` , reintento automático ante 401 (refresh + repetir), extracción de mensajes de error, `Idempotency-Key` en operaciones de dinero y normalización de listas paginadas (`asArray`).
- **Tipos generados** desde `contract/openapi.yaml` con `pnpm gen:api` (`schema.d.ts` no se edita a mano).
- **Pantallas**: landing, auth (login/registro/reset/verificación), dashboard, billetera, pagos, presupuestos, marketplace, ofertas, préstamos y detalle con pago de cuota, inversiones, perfil/KYC y Presti.
- **Avisos**: banner azul de demo (según `/meta/`), página legal y banner interno de compliance (oculto en demo).
- **Presti**: chat, mercado, noticias y marco legal; se puede apagar con `VITE_ENABLE_ASSISTANT=false` .
- **Mock**: `mock/server.mjs` levanta la API completa en memoria para trabajar sin backend (`pnpm mock` , puerto 8001).
- **Tests**: 37 unitarios con Vitest (formato de dinero, token en memoria, refresh, idempotencia, banner). Hay specs de Playwright (`e2e/`) que **no** corren en CI.

8. Infraestructura, deploy y CI

Docker

`Dockerfile` multi-stage:

- `base` : Python 3.14 slim, usuario `presto` , `WORKDIR /app` .
- `runtime` : dependencias **solo de producción**; el código se copia **después** de `pip install` para que un cambio de código no invalide la capa de dependencias; `collectstatic` ; `chmod` del entrypoint.
- `development` : agrega pytest, ruff, import-linter, factory-boy y pip-audit.
- `production` : etapa final (la que usa Render o `docker build .`).
- `deploy/entrypoint.sh` : si `RUN_MIGRATIONS=1` , aplica migraciones con reintentos y después ejecuta gunicorn (2 workers).
- `docker-compose.yml` : db, redis, web, worker y beat; puertos solo en localhost; `DEMO_MODE=true` .

- `docker-compose.prod.yml`: gunicorn, perfiles `proxy` (Caddy/TLS) y `backup`.

Render

`render.yaml` crea PostgreSQL, Key Value/Redis y el web. El web usa una **imagen preconstruida** de GHCR (`ghcr.io/jrweiss93/presto-mvp:latest`), no construye en Render. Variables clave: `RUN_MIGRATIONS=1`, `DEMO_MODE=true`, `CORS_ALLOWED_ORIGINS`, `FRONTEND_URL`, cookies `SameSite=None` / `Secure`, `ALLOWED_HOSTS` con `RENDER_EXTERNAL_HOSTNAME`. `initialDeployHook` siembra la demo en la primera creación.

Vercel

- Root Directory: `frontend`.
- `vercel.json`: rewrites de SPA para que las rutas profundas no den 404.
- Variables: `VITE_API_URL=https://<api>/api/v1`, `VITE_ENABLE_ASSISTANT=true`.

GitHub Actions

5 jobs: `quality` (ruff + import-linter), `migrations`, `tests` (PostgreSQL 16 + coverage), `frontend` (typecheck + tests + build) y `docker` (build + check de la imagen + publicación en GHCR en `main`).

9. Testing y calidad

- **800 tests** con pytest contra PostgreSQL real.
- `factory_boy` para usuarios, cuentas, préstamos, posiciones, etc.
- Settings de test: Celery en modo eager, email en memoria, depósitos simulados habilitados, `DEMO_MODE=False`.
- **Cobertura**: umbral global 80%; gates de 90% para `budgets`, `notifications` y `reports`.
- **import-linter**: 32 contratos de límites entre apps.
- **ruff**: lint + formato (línea 100, comillas simples).
- **CI**: `makemigrations --check`, `spectacular --validate --fail-on-warn`, `pip-audit`, `check --deploy` y build de la imagen.
- **Smoke test** (`scripts/smoke-demo.ps1`): health, modo demo, CORS, login, cookie, refresh, Presti y rutas del front (11 chequeos).
- **End-to-end**: `tests/test_end_to_end.py` recorre el flujo completo en ARS y USD y verifica el invariante de suma cero.

10. Decisiones de diseño (ADRs)

ADR	Tema
0001	Celery + Redis como cola y scheduler
0002	Presupuestos: el gasto se deriva del ledger
0003	Reportes de solo lectura
0004	Notificaciones por email
0005	Saldo desnormalizado
0006	Doble entrada con trigger diferido
0007	Orden de las apps
0008	Proveedor de pagos (Mercado Pago) detrás de una interfaz
0009	Cuenta segregada: Presto no custodia el dinero
0010	Plazos fijos sin garantía de capital
0011	PLA/FT mínimo viable
0012	Costo total y CFT
0013	Reintento de avisos por email
0014	Refresh token en cookie httpOnly
0015	Imágenes de producción fijadas a digest
0016	Reseteo de contraseña y verificación de email
0017	Presti: asistente por reglas, sin LLM

11. Qué está implementado y qué falta

Implementado

- Registro/login/logout, refresh rotativo, reseteo y verificación de email.
- Perfil y KYC (solicitud/aprobación manual).
- Ledger completo con idempotencia, reversos y triggers.
- Depósitos/retiros simulados + pagos con proveedor simulado o Mercado Pago.
- Marketplace de ofertas, comparación, préstamos con cronograma francés y pagos.
- Plazos fijos y liquidación automática.
- Presupuestos, gastos derivados y alertas.
- Cuatro reportes con CSV.
- Notificaciones por email con preferencias y reintentos.
- Presti sin LLM (mercado, noticias, legal y chat).
- Demo idempotente.
- Front completo, landing, legal y aviso de demo.
- Docker, Render, Vercel, GHCR y CI.

Pendiente / límites

- Contrato y credenciales reales de pagos; hoy el dinero es simulado.
- KYC con subida de documentos o proveedor de identidad.
- TyC, privacidad y encuadre regulatorio definitivo (CNV/BCRA/PLA-FT).
- SMTP real para emails (en demo van a consola).
- Sentry, backups automáticos probados y alertas de infraestructura.

- E2E del front en CI y auditoría de accesibilidad.
- Presti con LLM (hoy es por reglas, a propósito).

12. Mapa rápido y comandos

Archivos clave

```
manage.py / config/settings/       configuración
apps/core/money.py                 dinero y amortización
apps/ledger/services.py            movimientos del libro mayor
apps/payments/providers.py         simulado + Mercado Pago
apps/loans/services.py             préstamos y cuotas
apps/budgets/selectors.py          regla de gasto
apps/assistant/services.py         respuestas de Presti
frontend/src/shared/api/client.ts  sesión e idempotencia
frontend/src/app/App.tsx           rutas
Dockerfile / deploy/entrypoint.sh  imagen y migraciones
render.yaml / frontend/vercel.json deploy
```

Comandos útiles

```
pytest -q
ruff check . && ruff format --check .
lint-imports
python manage.py makemigrations --check --dry-run
python manage.py spectacular --validate --fail-on-warn --file /tmp/schema.yml
python manage.py seed_demo
python manage.py check --deploy
```

En Windows, con el entorno nativo:

```
.\scripts\start-services.ps1
. .\scripts\env.ps1
.\.venv\Scripts\python.exe -m pytest -q
.\.venv\Scripts\python.exe manage.py runserver
```